

1、进程管理

1. Describe the concepts of operation system
2. Multiprogramming, multitasking (time sharing)
3. Time share system, real-time system
4. Network operating system, distributed operating system
5. Compare the concepts of concurrency and parallelism.
6. What is dual-mode of execution in OS? What are its benefits?
7. Describe the differences between symmetric and asymmetric multiprocessing. What are three advantages and one disadvantage of multiprocessor systems?
8. Give two reasons why caches are useful. What problems do they solve?
9. What problems do they cause? If a cache can be made as large as the device for which it is caching (for instance, a cache as large as a disk), why not make it that large and eliminate the device?
10. Define the essential properties of the following types of operating systems:
 - a. Batch
 - b. Interactive
 - c. Time sharing
 - d. Real time
 - e. Network
 - f. Parallel
 - g. Distributed
 - h. Clustered
 - i. Handheld
11. Describe three general methods for passing parameters to the operating system.
12. What is the main advantage of the microkernel approach to system design?
How do user programs and system services interact in a microkernel architecture?
What are the disadvantages of using the microkernel approach?
13. In what ways is the modular kernel approach similar to the layered approach? In what ways does it differ from the layered approach?
14. What is the purpose of interrupts? How does an interrupt differ from a trap? Can traps be generated intentionally by a user program? If so, for what purpose?
15. What is the purpose of system calls?
16. What are the three main purposes of an operating system?
17. We have stressed the need for an operating system to make efficient use of the computing hardware. When is it appropriate for the operating system to forsake this principle and to “waste” resources? Why is such a system not really wasteful?
18. The issue of resource utilization shows up in different forms in different types of operating systems. List what resources must be managed carefully in the following settings:
 - a. Mainframe or minicomputer systems
 - b. Workstations connected to servers
 - c. Mobile computers

19. Under what circumstances would a user be better off using a timesharing system than a PC or a single-user workstation?

20. Many SMP systems have different levels of caches; one level is local to each processing core, and another level is shared among all processing cores. Why are caching systems designed this way?

21. The services and functions provided by an operating system can be divided into two main categories. Briefly describe the two categories, and discuss how they differ.

22. Would it be possible for the user to develop a new command interpreter using the system-call interface provided by the operating system?

23. Why is the separation of mechanism and policy desirable?

24. 内核模式和用户模式之间如何区分，如何作为保护（安全）的基本形式？

答案：内核模式和用户模式之间的区别通过以下方式提供了一种基本的保护形式。某些指令只能在 CPU 处于内核模式时执行。类似地，只有当程序处于内核模式时才能访问硬件设备，并且只有在 CPU 处于内核模式时才能启用或禁用中断。因此，CPU 在用户模式下执行时的能力非常有限，从而实现了对关键资源的强制保护。

25. 进程间通信的两个模型是什么？这两种方案有何优缺点？

26. 采用可加载内核模块的优点有什么？

27. Describe the differences between process and program.

28. Describe the actions taken by a kernel to context-switch between processes.

29. The Fibonacci sequence... Write a C program using the fork() system call that generates the Fibonacci sequence in the child process.

30. Under what circumstances does a multithreaded solution using multiple kernel threads provide better performance than a single-threaded solution on a single-processor system?

31. Which of the following components of program state are shared across threads in a multithreaded process?

- a. Register values
- b. Heap memory
- c. Global variables
- d. Stack memory

32. In multithreading system, threads in process P *cannot* share:、

- A. Code
- B. Files
- C. Global variables
- D. Stack of a thread

33. When a process creates a new process using the fork() operation, which of the following states is shared between the parent process and the child process?

- 1. Stack

2. Heap

3. Shared memory segments

34. What are two differences between user-level threads and kernel-level threads? Under what circumstances is one type better than the other?

35. 要求如下:

已知从这个程序执行到这个程序的所有进程结束这个时间段内,没有其它新进程执行。

```
#include "stdio.h"
#include "sys/types.h"
#include "unistd.h"
int main() {
    pid_t pid1;
    pid_t pid2;
    pid1 = fork();
    pid2 = fork();
    printf("pid1:%d, pid2:%d\n", pid1, pid2);
}
```

1、请说出执行这个程序后,将一共运行几个进程。

2、如果其中一个进程的输出结果是“pid1:1001, pid2:1002”,写出其他进程的输出结果(不考虑进程执行顺序),并加以解释。

36. Consider a multiprogramming environment, two programs A and B share the system simultaneously, and run as follows.

(1) For every 25 min, A runs on CPU for 15 min, then waits for I/O for 10 mi

For every 25 min, B runs on CPU for 20 min, then waits for I/O for 5 min.

Suppose A runs first, I/OA and I/OB are different devices, and the time of switching between A and B can be ignored. Draw the timeline for these two programs, and calculate CPU utilization within 60 minutes.

(2) What if I/OA and I/OB are the same device?

37. Draw the diagram of process state. Mark the corresponding events that might cause each transition.

38. In the following program, How many processes will be created?

(a)

```
#include <unistd.h>
#include <stdio.h>
int main(){
    pid_t pid;  int a=5, b=7;
    pid = fork();      /* fork another process */
    if (pid < 0) {      /* error occurred */
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) { /* child process */
```

```

        a=a*2;
        printf ("this is Child process, a=%d, b=%d\n",a,b);
    }
    else {
        /* parent process */
        printf ("this is parent process, a=%d, b=%d\n",a,b);
        wait (NULL);/* parent will wait for the child to
complete */
        a=a+1; b=b+3;
        printf ("Child Complete, a=%d, b=%d\n",a,b);
        exit(0);
    }
}

```

(b) (optional)

```

#include <unistd.h>
#include <stdio.h>
int main()
{
    fork();
    fork() && fork() || fork();
    fork();
    sleep(100);
    return 0;
}

```

39. Provide two programming examples in which multithreading does not provide better performance than a single-threaded solution.

40. Can a multithreaded solution using multiple user-level threads achieve better performance on a multiprocessor system than on a single-processor system? Explain.

41. Is it possible to have concurrency but not parallelism? Explain.

42. Using Amdahl's Law, calculate the speedup gain for the following applications:

- 40 percent parallel with (a) eight processing cores and (b) sixteen processing cores
- 67 percent parallel with (a) two processing cores and (b) four processing cores
- 90 percent parallel with (a) four processing cores and (b) eight processing cores

43. Consider a multicore system and a multithreaded program written using the many-to-many threading model. Let the number of user-level threads in the program be greater than the number of processing cores in the system. Discuss the performance implications of the following scenarios.

- a. The number of kernel threads allocated to the program is less than the number of processing cores.
- b. The number of kernel threads allocated to the program is equal to the number of processing cores.

c. The number of kernel threads allocated to the program is greater than the number of processing cores but less than the number of user-level threads.

44. 采用图示的程序，确定行 A、B、C、D 中的 pid 的值（假定父进程和子进程的 pid 分别为 2600 和 2603）

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid, pid1;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        pid1 = getpid();
        printf("child: pid = %d", pid); /* A */
        printf("child: pid1 = %d", pid1); /* B */
    }
    else { /* parent process */
        pid1 = getpid();
        printf("parent: pid = %d", pid); /* C */
        printf("parent: pid1 = %d", pid1); /* D */
        wait(NULL);
    }

    return 0;
}
```

45. 编写一个名为 time.c 的 C 程序，以确定从命令行运行命令所需的时间量。该程序将作为 “./time <command>” 运行，并报告运行指定命令所用的时间量。这将涉及使用函数 fork() 和 exec()，以及函数 gettimeofday() 来确定经历的时间。它还需要使用两种不同的 IPC 机制。

一般策略是派生一个将执行指定命令的子进程。然而，在子进程执行命令之前，它会记录当前时间的时间戳（我们称之为“开始时间”）。父进程将等待子进程终止。一旦子进程终止，父进程将记录结束时间的当前时间戳。开始时间和结束时间

之间的差异表示执行命令所经历的时间。下面的示例输出报告了运行命令 `ls` 的时间量：

```
./time ls
time.c
time
:
Elapsed time 0.25422
```

由于父进程和子进程是独立的进程，因此它们需要安排如何在它们之间共享开始时间。你将编写该程序的两个版本，每个版本代表不同的 IPC 方法。

第一个版本让子进程在调用 `exec()` 之前将开始时间写入共享内存区域。子进程终止后，父进程会从共享内存中读取开始时间。有关使用 POSIX 共享内存的详细信息，请参阅 3.7.1 节。在该部分中，生产者和消费者有单独的程序。由于解决这个问题只需要一个程序，因此可以在子进程被复刻 (`fork`) 之前建立共享内存区域，从而允许父进程和子进程访问共享内存区域。

第二个版本使用管道。子进程将开始时间写入管道，父进程将在子进程终止后从中读取。

你将使用 `gettimeofday()` 函数记录当前时间戳。该函数传递一个指向 `struct timeval` 对象的指针，该对象包含两个成员：`tv_sec` 和 `tv_usec`。它们表示自 1970 年 1 月 1 日以来经过的秒数 and 微秒数（称为 UNIX EPOCH）。以下代码示例说明了如何使用此函数：

```
struct timeval current;
gettimeofday(&current, NULL);
// current.tv_sec represents seconds
// current.tv_usec represents microseconds
```

对于子进程和父进程之间的 IPC，可以为共享内存指针的内容分配表示开始时间的 `struct timeval`。使用管道时，可以将指向 `struct timeval` 的指针写入，并从管道中读取。

46. 具有两个双核处理器的系统有四个处理核可用于调度。这个系统有一个 CPU 密集型应用程序运行。在程序启动时，所有输入通过打开一个文件而读入。同样，在程序终止之前，所有程序输出结果都写入一个文件。在程序启动和终止之间，该程序为 CPU 密集型的。你的任务是通过多线程技术来提高这个应用程序的性能。

47. 这个应用程序运行在采用一对一线程模型的系统中（每个用户线程映射到一个内核线程）。

48. 你将创建多少个线程用于执行输入和输出？请给出解释。

49. 你将创建多少个线程用于应用程序的 CPU 密集型部分？请给出解释。

50. 考虑下面的代码段：

```
pid_t pid;
pid = fork();
if (pid == 0) { /* child process */
    fork();
    thread_create( ... );
}
fork();
```

- a. 创建了多少个单独进程？
- b. 创建了多少个单独线程？

51. 如图 4.23 所示的程序采用 Pthreads API。该程序的 LINE_C 和 LINE_P 的输出分别是什么？

```
#include <pthread.h>
#include <stdio.h>

int value = 0;
void *runner(void *param); /* the thread */

int main(int argc, char *argv[])
{
    pid_t pid;
    pthread_t tid;
    pthread_attr_t attr;

    pid = fork();

    if (pid == 0) { /* child process */
        pthread_attr_init(&attr);
        pthread_create(&tid, &attr, runner, NULL);
        pthread_join(tid, NULL);
        printf("CHILD: value = %d", value); /* LINE C */
    }
    else if (pid > 0) { /* parent process */
        wait(NULL);
        printf("PARENT: value = %d", value); /* LINE P */
    }
}

void *runner(void *param)
{
    value = 5;
    pthread_exit(0);
}
```

52. What is the output?

```
int value = 5;
int main(int argc, char *argv[]) {
    pid_t pid;
    pid = fork();
    if (pid == 0) {
        printf("child process, value1 : %d\n", value);
        value += 15;
        printf("child process, value2 : %d\n", value);
    }
    else if (pid > 0) {
        printf("parent process, value3 : %d\n", value);
        wait(NULL);
        printf("parent process, value4 : %d\n", value);
    }
    exit(0);
}
```

答案:

```
child process, value1 : 5
child process, value2 : 20
parent process, value3 : 5
parent process, value4 : 5

parent process, value3 : 5
child process, value1 : 5
child process, value2 : 20
parent process, value4 : 5
```

53. Including the initial parent process, how many processes are created by the following program.

```
#include <stdio.h>
#include <unistd.h>
int main() {
    int i;
    for (i = 0; i < 4; i++)
        fork();
    return 0;
}
```

54. Draw the diagram of process state.

```
/* kai.c */
#include <stdio.h>
#include <pthread.h>

void *helloFunc(void *ptr) {
```



```

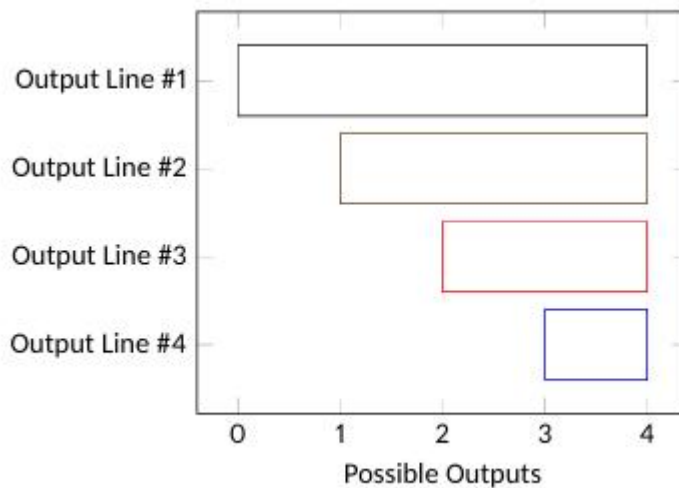
    int *data;
    data = (int *) ptr;
    printf("I'm Thread %d\n", *data);
    return (void *) data;
}

int main(int argc, char *argv[]) {
    pthread_t hThread[4];
    int *rvals[4];
    for (int i = 0; i < 4; i++) {
        pthread_create(&hThread[i], NULL, helloFunc, (void *)
&i);
    }
    for (int i = 0; i < 4; i++) {
        pthread_join(hThread[i], (void **)&rvals[i]);
        //printf("Thread %d returns %d\n", i, *rvals[i])
    }
    return 0;
}

```

答案:

Any possible order of a non-decreasing sequence satisfying ...



2、CPU 调度

1. Describe the differences among short-term, medium-term, and long-term scheduling.
2. Why is it important for the scheduler to distinguish I/O-bound programs from CPU-bound programs?
3. Discuss how the following pairs of scheduling criteria conflict in certain settings.
 - a. CPU utilization and response time

- b. Average turnaround time and maximum waiting time
- c. I/O device utilization and CPU utilization

4. Consider the following set of processes, with the length of the CPU-burst time given in milliseconds:

Process	Burst Time	Priority
P ₁	10	3
P ₂	1	1
P ₃	2	3
P ₄	1	4
P ₅	5	2

The processes are assumed to have arrived in the order P₁, P₂, P₃, P₄, P₅, all at time 0.

- a. Draw four Gantt charts illustrating the execution of these processes using FCFS, SJF, a nonpreemptive priority (a smaller priority number implies a higher priority), and RR (quantum = 1) scheduling.
- b. What is the turnaround time of each process for each of the scheduling algorithms in part a?
- c. What is the waiting time of each process for each of the scheduling algorithms in part a?
- d. Which of the schedules in part a results in the minimal average waiting time (over all processes)?

5. Which of the following scheduling algorithms could result in starvation?

- a. First-come, first-served
- b. Shortest job first
- c. Round robin
- d. Priority

6. Explain the differences in the degree to which the following scheduling algorithms discriminate in favor of short processes:

- a. FCFS
- b. RR
- c. Multilevel feedback queues

7. Consider the following set of processes, assumed to have arrived at time 0, in the order P₁, P₂, P₃, P₄, P₅, with the length of the CPU-burst time given in milliseconds:

(a) Draw four Gantt charts that illustrate the execution of these processes using FCFS, non-preemptive SJF, non-preemptive priority (a smaller priority number implies a higher priority), and RR (quantum = 1) scheduling.

(b) What are the average turnaround time and the average waiting time of all process for each of the scheduling algorithms in part (a) respectively?

8. Suppose that the following processes arrive for execution at the times indicated. Each process will run for the amount of time listed. In answering the questions, use nonpreemptive scheduling, and base all decisions on the information you have at the time the decision must be made.

Process	Arrival Time	Burst Time
P1	0.0	8
P2	0.4	4
P3	1.0	1

1. What is the average turnaround time for these processes with the FCFS scheduling algorithm?
2. What is the average turnaround time for these processes with the SJF scheduling algorithm?
3. The SJF algorithm is supposed to improve performance, but notice that we chose to run process P1 at time 0 because we did not know that two shorter processes would arrive soon. Compute what the average turnaround time will be if the CPU is left idle for the first 1 unit and then SJF scheduling is used. Remember that processes P1 and P2 are waiting during this idle time, so their waiting time may increase. This algorithm could be called future-knowledge scheduling.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	
FCFS:	P ₁							P ₂			P ₃				
SJF:	P ₁							P ₃	P ₂						
SJF':		P ₃	P ₂				P ₁								

$$1. T_{\text{turnaround}} = \frac{8 + (12 - 0.4) + (13 - 1)}{3} \approx 10.53$$

$$2. T_{\text{turnaround}} = \frac{8 + (13 - 0.4) + (9 - 1)}{3} \approx 9.53$$

$$3. T_{\text{turnaround}} = \frac{14 + (6 - 0.4) + (2 - 1)}{3} \approx 6.87$$

9. The following processes are being scheduled using a preemptive, roundrobin scheduling algorithm. Each process is assigned a numerical priority, with a higher number indicating a higher relative priority. In addition to the processes listed below, the system also has an idle task (which consumes no CPU resources and is identified as P_idle). This task has priority 0 and is scheduled whenever the system has no other

available processes to run. The length of a time quantum is 10 units. If a process is preempted by a higher-priority process, the preempted process is placed at the end of the queue.

Thread	Priority	Burst	Arrival
P1	40	20	0
P2	30	25	25
P3	30	25	30
P4	35	15	60
P5	5	10	100
P6	10	10	105

1. Show the scheduling order of the processes using a Gantt chart.
2. What is the turnaround time for each process?
3. What is the waiting time for each process?
4. What is the CPU utilization rate?

1. Gantt chart:

	5	10	15	20	25	30	35	40	45	50	55	60	65	70	75	80	85	90	95	100	105	110	115	120
RR:	P ₁	P _{idle}	P ₂	P ₃	P ₂	P ₃	P ₄	P ₂	P ₃	P _{idle}	P ₅	P ₆	P ₅											

2. $T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$

3. $T_{\text{waiting}} = T_{\text{turnaround}} - T_{\text{burst}}$

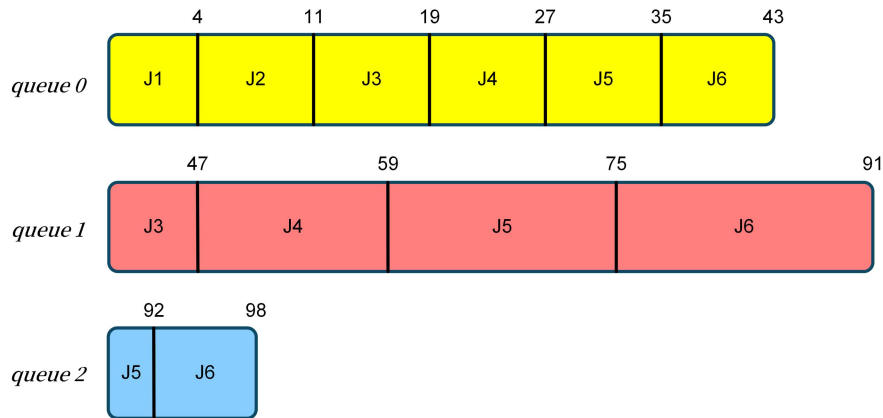
	P ₁	P ₂	P ₃	P ₄	P ₅	P ₆
$T_{\text{turnaround}}$	20-0=20	80-25=55	90-30=60	75-60=15	120-100=20	115-105=10
T_{waiting}	20-20=0	55-25=30	60-25=35	15-15=0	20-10=10	10-10=0

4. $\text{rate} = \frac{105}{120} \approx 87.50\%$

10. CPU Scheduling [10 pts] (Answer this question on the answer sheet.)

Consider a multi-level feedback queue in a single-CPU system. The first level (queue 0) is given a quantum of 8 ms, the second one a quantum of 16 ms, the third is scheduled FCFS. Assume jobs arrive all at time zero with the following job times (in ms): 4, 7, 12, 20, 25 and 30, respectively. Assume the context switch overhead is zero unless otherwise stated.

- (1) Show the Gantt chart for this system.
- (2) Compute the average waiting and turnaround time.
- (3) Suppose the context switch overhead is 1 ms. Compute the average turnaround time.



(b) 平均等待时间: $(0 + 4 + (11 + 43 - 19) + (19 + 47 - 27) + (27 + 59 - 35 + 91 - 75) + (35 + 75 - 43 + 92 - 91)) / 6 = 35.5\text{ms}$ 平均周转时间: $(4 + 11 + 47 + 59 + 92 + 98) / 6 = 311 / 6 = 51.83\text{ms}$

(c) 平均周转时间: $(4 + 12 + 53 + 66 + 102 + 109) / 6 = 57.67\text{ms}$

11. Consider the following set of processes with arrival time, burst time, and priority:

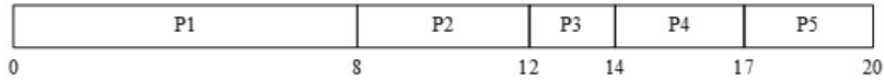
Process	Burst Time	Arrival Time	Priority
P ₁	8	0	10
P ₂	4	1	2
P ₃	2	3	10
P ₄	3	5	1
P ₅	3	7	5

Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithms:

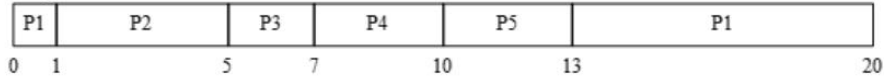
- FCFS
- preemptive SJF (FCFS if remain burst time is equal)
- preemptive priority (FCFS if priority is equal)

- RR (quantum = 2)

(1). FCFS:



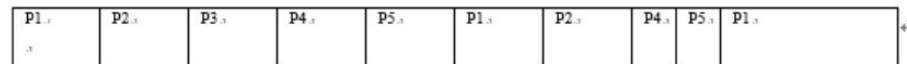
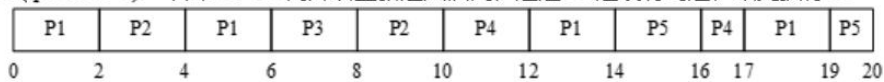
Preemptive SJF:



Preemptive priority:



RR(quantum=2): 答案 1: P1 离开后重新进入队列, 这是 P3 还没有到达, 故先执行 P1

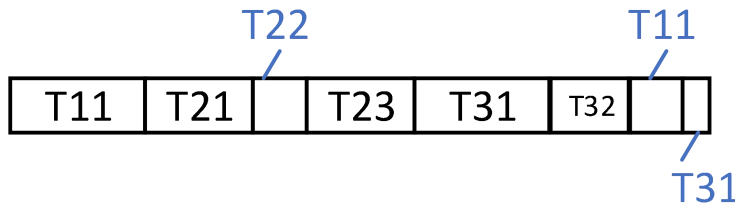
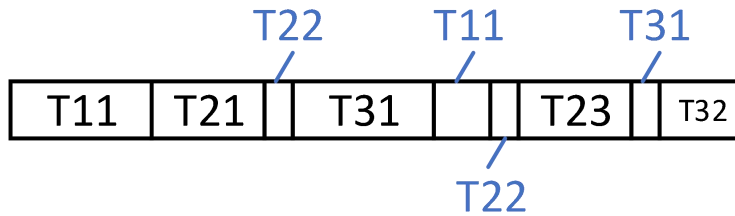


12. 假定有三个进程 (P1, P2, P3), P1 有 1 个线程 T11, P2 有 3 个线程 T21, T22 和 T23, P3 有 2 个线程 T31 和 T32。这些线程的 CPU 运行时间如下表所示。假设所有线程同时到达, 采用时间片轮转调度算法, 时间片为 5, 画出下列两种条件的甘特图, 计算每一个进程的等待时间和周转时间, 并计算其平均等待时间和平均周转时间。

(1) 线程是用户级线程

(2) 线程是内核级线程

进程	线程	运行时间
P1	T11	7
P2	T21	4
	T22	2
	T23	4
P3	T31	6
	T32	3



13. Consider five processes: P1, P2, P3, P4, P5, with the length of the CPU-burst time given in milliseconds: [12 marks] (Please write the answers on the answer sheet.)
 (a) All processes assumed to have arrived at time 0, in the order P1, P2, P3, P4, P5. Draw four Gantt charts that illustrate the execution of these processes using FCFS, non-preemptive SJF, non-preemptive priority (a smaller priority number implies a higher priority), and RR (quantum = 2) scheduling. What are the average turnaround time and the average waiting time of all process for each of the scheduling algorithms respectively?

Process	P1	P2	P3	P4	P5
Burst Time	9	3	2	1	5
Priority	3	1	5	4	2

(b) All processes P1, P2, P3, P4, P5 assumed arriving at time 1, 3, 5, 7, 9. Draw four Gantt charts that illustrate the execution of these processes using FCFS, non-preemptive SJF, non-preemptive priority (a smaller priority number implies a higher priority), and RR (quantum = 2) scheduling. What are the average turnaround time and the average waiting time of all process for each of the scheduling algorithms respectively?

Process ↴	P1 ↴	P2 ↴	P3 ↴	P4 ↴	P5 ↴
Burst Time ↴	9 ↴	3 ↴	2 ↴	1 ↴	5 ↴
Priority ↴	3 ↴	1 ↴	5 ↴	4 ↴	2 ↴
Arriving Time ↴	1 ↴	3 ↴	5 ↴	7 ↴	9 ↴

Process ↴	P1 ↴	P2 ↴	P3 ↴	P4 ↴	P5 ↴
Burst Time ↴	9 ↴	3 ↴	2 ↴	1 ↴	5 ↴
Priority ↴	3 ↴	1 ↴	5 ↴	4 ↴	2 ↴
I/O ↴	1.5ms IO/after 2ms CPU ↴	no ↴	no ↴	no ↴	no ↴

14. Consider the exponential average formula used to predict the length of the next CPU burst. What are the implications of assigning the following values to the parameters used by the algorithm?

- $\alpha = 0$ and $\tau_0 = 100$ milliseconds
- $\alpha = 0.99$ and $\tau_0 = 10$ milliseconds

15. 假设有如下一组进程，它们的 CPU 突发时间以毫秒来计算：

进程	突发时长	优先级
P1	5	4
P2	3	1
P3	1	2
P4	7	2
P5	4	3

假设进程按 P1、P2、P3、P4、P5 顺序在时刻 0 到达。

- 画出 4 个 Gantt 图，分别演示采用每种调度算法（FCFS、SJF、非抢占优先级（一个较大优先级数值意味着更高优先级）和 RR（时间额 = 2））的进程执行。
- 每个进程在 a 里的每种调度算法下的周转时间是多少？
- 每个进程在 a 里的每种调度算法下的等待时间是多少？
- 哪一种调度算法的平均等待时间（对所有进程）最小？

16. 以下进程使用抢占式、基于优先级的循环调度算法进行调度。每个进程都分配有一个数字优先级，数字越大表示相对优先级越高。调度程序将执行最高优先级的

进程。对于具有相同优先级的进程，将使用循环调度程序，时间配额为 10 个单位。如果一个进程被更高优先级的进程抢占，则被抢占的进程被放置在队列的末尾。

进程	优先级	突发时长	到达时间
P1	8	15	0
P2	3	20	0
P3	4	20	20
P4	4	20	25
P5	5	5	45
P6	5	15	55

- 使用 Gantt 图显示进程的调度顺序。
- 每个进程的周转时间是多少？
- 每个进程的等待时间是多少？

17. [408]（本题满分 7 分）系统采用优先级（优先级越大表示优先级越高）与时间片轮转调度算法，仅当发生时钟中断时才触发抢占 CPU 操作，时钟中断间隔为 10 ms。进程首次进入就绪队列时，其时间片为 50 ms。若进程因时间片用完而返回就绪队列，其优先级值减 1；若进程被更高优先级进程抢占而返回就绪队列，其优先级值保持不变。当多个进程优先级相同时，先进入就绪队列的进程优先被调度。四个进程的到达时刻、初始优先级与 CPU 运行时间如下表所示：

进程	到达就绪队列时间 (ms)	优先级	CPU 运行时间 (ms)
P1	10	3	95
P2	10	4	20
P3	12	2	40
P4	14	5	60

（1）从 10 ms 开始进程调度，直至所有进程调度结束，此时中断次数与 CPU 调度次数分别为多少？P1、P2、P3、P4 各自的首次调度发生在哪个时刻？（5 分）

（2）若时间片由 50 ms 改为 100 ms，CPU 调度次数将增大、不变还是减少？若时钟中断间隔由 10 ms 改为 1 ms，系统开销将增大、不变还是减少？（2 分）

24. Consider the following set of processes, with the length of the CPU burst given in milliseconds:

Process	Burst Time	Priority
P1	2	2

Process	Burst Time	Priority
P2	1	1
P3	8	4
P4	4	2
P5	5	3

The processes are assumed to have arrived in the order of P1, P2, P3, P4, P5, all at time 0.

1. Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithms: FCFS, SJF, non-preemptive priority (a larger priority number implies a higher priority), and RR (quantum = 2).
2. What is the average turnaround time for each of the scheduling algorithms in part 1?
3. What is the waiting time of each process for each of these scheduling algorithms?
4. Which of the algorithms results in the minimum average waiting time (over all processes)?

答案:

Q1:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
FCFS	P ₁	P ₂	P ₃									P ₄				P ₅				
SJF	P ₂	P ₁	P ₄				P ₅					P ₃								
PRIQ	P ₃							P ₅					P ₁		P ₄			P ₂		
RR	P ₁	P ₂	P ₃	P ₄		P ₅		P ₃		P ₄		P ₅		P ₃		P ₅		P ₃		

Q2:

	FCFS	SJF	PRIQ	RR
$T_{\text{turnaround}} \text{ (ms)}$	$\frac{2+3+11+15+20}{5}$ =10.2	$\frac{1+3+7+12+20}{5}$ =8.6	$\frac{8+13+15+19+20}{5}$ =15	$\frac{2+3+20+13+18}{5}$ =11.2

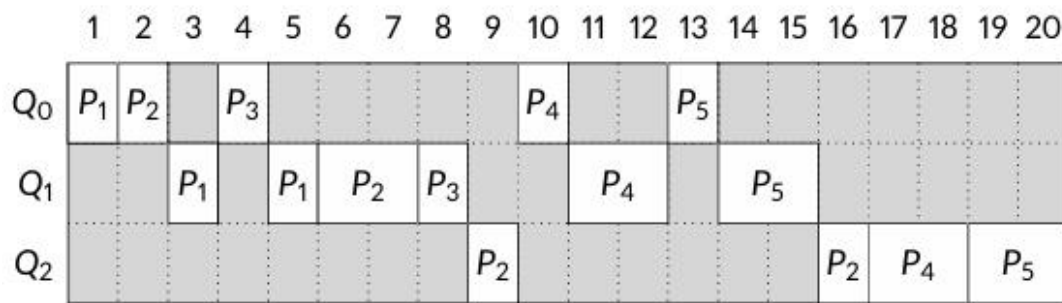
Q3 & Q4:

		P ₁	P ₂	P ₃	P ₄	P ₅	Avg
$T_{\text{waiting}} \text{ (ms)}$	FCFS	0	2	3	11	15	6.2
	SJF	1	0	12	3	7	4.8
	PRIQ	13	19	0	15	8	11
	RR	0	2	12	9	13	7.2

25. Given the table below showing the process information of a system, based on multilevel feedback queuing scheduling scheme. Suppose there are five levels in the system and within each level the FCFS scheduling is used. If a process is preempted, it will wait for being scheduled at the first place in the FCFS queue. Draw a Gantt chart to show the time of CPU allocated to each process until all processes are finished using time quantum $q = 2^i$ (where q = time allocated for a process to run in its turn, and i ranges from 0 to 4 indicating the i^{th} level queue).

Process	Arrival Time	Service Time
P1	0	3
P2	1	5
P3	3	2
P4	4	9
P5	5	12

答案:



3、进程同步与死锁

1. What is the meaning of the term *busy waiting*? What other kinds of waiting are there in an operating system? Can busy waiting be avoided altogether? Explain your answer.
2. Explain why spinlocks are not appropriate for single-processor systems yet are often used in multiprocessor systems.
3. Describe how the Swap() instruction can be used to provide mutual exclusion that satisfies the bounded-waiting requirement.
4. Servers can be designed to limit the number of open connections. For example, a server may wish to have only N socket connections at any point in time. As soon as N connections are made, the server will not accept another incoming connection until an existing connection is released. Explain how semaphores can be used by a server to limit the number of concurrent connections.
5. Show that, if the wait() and signal() semaphore operations are not executed atomically, then mutual exclusion may be violated.
6. The Sleeping-Barber Problem. A barbershop consists of a waiting room with n chairs and a barber room with one barber chair. If there are no customers to be served, the barber goes to sleep. If a customer enters the barbershop and all chairs are occupied, then

the customer leaves the shop. If the barber is busy but chairs are available, then the customer sits in one of the free chairs. If the barber is asleep, the customer wakes up the barber. Write a program to coordinate the barber and the customers.

7. 假定一个阅览室最多可同时容纳 n 个人阅读，读者进入和离开阅览室时，都必须在阅览室门口的一个登记表上登记。假定每次只允许一个人登记和注销。请用 PV 操作编写读者进程的同步算法，并加以解释

```
int seats=N; semaphore mutex (1);
reader(){
    P(mutex);
    If (seats>0){
        seats--;
        login();
        V(mutex);

        reading();    //进入阅览室阅读

        P(mutex);
        seats++;
        logout();
        V(mutex);
    }
    else V(mutex);    //离开
}
```

8. One teacher and fifty students participate in the midterm exam. Only one person can enter or leave the examination room at one time. The teacher can not distribute the exam papers until fifty students all enter the examination room. The students can leave the examination room after they hand in their examination papers. The teacher can not leave the examination room until he collects all the papers.

9. Consider the deadlock situation that could occur in the dining-philosophers problem when the philosophers obtain the chopsticks one at a time. Discuss how the four necessary conditions for deadlock indeed hold in this setting. Discuss how deadlocks could be avoided by eliminating any one of the four conditions.

10. In a real computer system, neither the resources available nor the demands of processes for resources are consistent over long periods (months). Resources break or are replaced, new processes come and go, new resources are bought and added to the system. If deadlock is controlled by the banker's algorithm, which of the following changes can be made safely (without introducing the possibility of deadlock), and under what circumstances?

- a. Increase Available (new resources added).
- b. Decrease Available (resource permanently removed from system).
- c. Increase Max for one process (the process needs more resources than allowed, it may want more).
- d. Decrease Max for one process (the process decides it does not need that many resources).
- e. Increase the number of processes.

f. Decrease the number of processes.

11. Consider the following snapshot of a system. Answer the following questions using the banker's algorithm:

	Allocation	Max	Available
	A B C D	A B C D	A B C D
P ₀	0 0 1 2	0 0 1 2	1 5 2 0
P ₁	1 0 0 0	1 7 5 0	
P ₂	1 3 5 4	2 3 5 6	
P ₃	0 6 3 2	0 6 5 2	
P ₄	0 0 1 4	0 6 5 6	

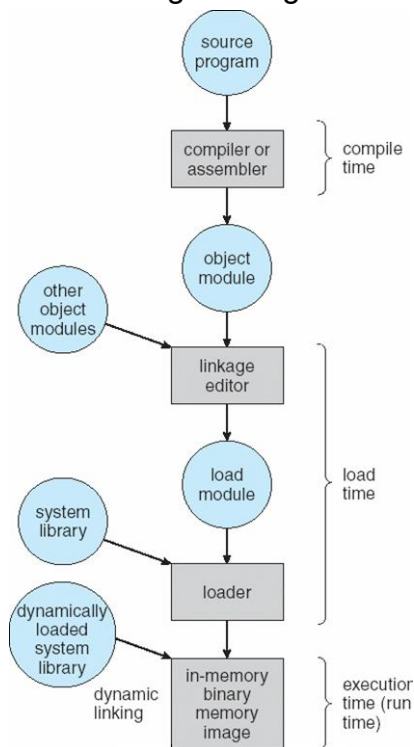
- What is the content of the matrix *Need*?
- Is the system in a safe state?
- If a request from process P₁ arrives for (0,4,2,0), can the request be granted immediately?

12. Consider the following system snapshot using the data structures in the Banker's algorithm, with resources A, B, C, and processes P1 to P5:

Processes	<i>Allocation</i>			<i>MAX</i>			<i>Need</i>			<i>Available</i>		
	A	B	C	A	B	C	A	B	C	A	B	C
P ₁	1	2	0	3	9	0						
P ₂	0	2	1	1	6	3						
P ₃	1	1	1	2	4	2						
P ₄	0	1	2	0	6	3						
P ₅	2	1	0	5	4	3						
Total										1	3	2

- What are the contents of the Need matrix? Fill in the above table.
- Is the system in a safe state? Provide your reasons.
- If a request from process P5 arrives for additional resources of (0,0,1), can the Banker's algorithm grant the request immediately?
- Based on the system state in the answer to question (3), if another request from process P3 arrives for additional resources of (0,0,1), can the Banker's algorithm grant the request immediately?

5. Then based on the system state in the answer to question (4), if another request from process P1 arrives for additional resources of (0,0,1), can the Banker's algorithm grant the request immediately?



13. The first known correct software solution to the critical-section problem for two processes was developed by Dekker. The two processes, P0 and P1, share the following variables:

boolean flag[2]; /* initially false */ int turn;

The structure of process P_i (i == 0 or 1) is shown in the following Figure.

Prove that the algorithm satisfies all three requirements for the critical-section problem.

```

while (true) {
    flag[i] = true;
    while (flag[j]) {
        if (turn == j) {
            flag[i] = false;
            while (turn == j)
                ; /* do nothing */
            flag[i] = true;
        }
    }
    /* critical section */
    turn = j;
    flag[i] = false;
    /* remainder section */
}
  
```

}

14. Explain why implementing synchronization primitives by disabling interrupts is not appropriate in a singleprocessor system if the synchronization primitives are to be used in user-level programs.

15. Explain why interrupts are not appropriate for implementing synchronization primitives in multiprocessor systems.

16. Problem Description:

17. Imagine there's an online food delivery platform that has several chefs (producers) and delivery workers (consumers). Each chef can prepare one dish at a time, and each delivery worker can deliver the prepared food to customers. The delivery platform has the following constraints and requirements:

- a)** Each chef can only prepare one dish at a time and must place it into one of the platform's delivery boxes after preparation.
 - b)** Each delivery box can only hold one dish.
 - c)** Delivery workers can only take food from the delivery boxes to deliver.
 - d)** If a delivery box is empty, the delivery worker must wait until there is food available to deliver.
 - e)** To ensure food freshness, chefs cannot prepare new food if the delivery box is full. They must wait for a delivery worker to take the food before starting the next dish.
 - f)** There are multiple delivery boxes on the platform, but each chef and delivery worker can only access one delivery box.
 - g)** Synchronization Requirements:
 - h)** It is necessary to ensure that chefs do not prepare food when the delivery box is full.
 - i)** Delivery workers cannot take food from an empty delivery box.
 - j)** A delivery box cannot be accessed by multiple chefs or delivery workers at the same time.
- Using semaphore to solve the problem.

18. Consider a concurrent program that includes two functions, called funcA and funcB. This program has the following synchronization requirements, both of which must be satisfied.

- a)** Requirement 1: At most one thread at a time may be running funcB.
- b)** Requirement 2: At most two threads at a time may be running any combination of funcA or funcB.

These requirements are summarized in the table, which shows which combinations of funcA and funcB may be executed concurrently. Note that it is

never OK for more than two threads to be running any combination of these functions concurrently.

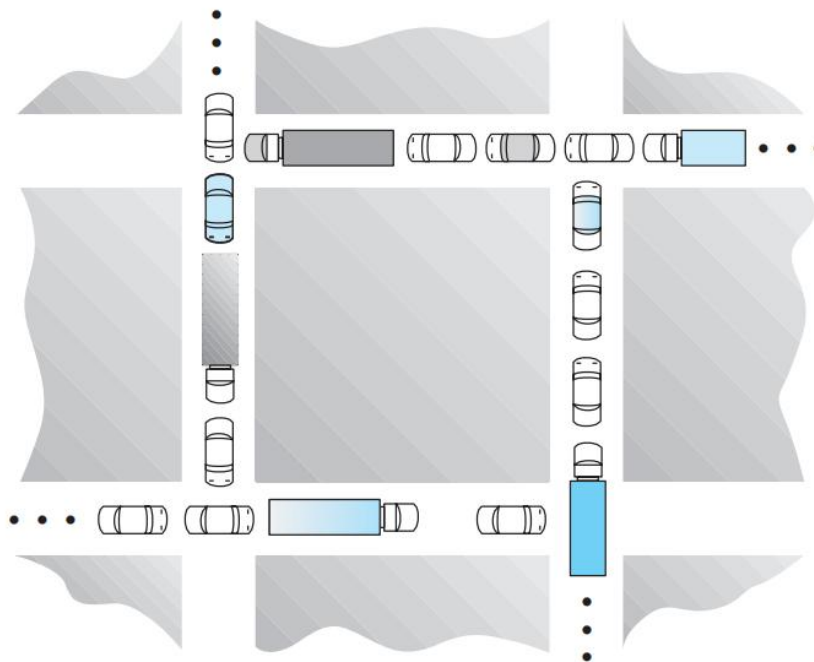
	funcA	funcB
funcA	OK	OK
funcB	OK	NO

Fig. 1 Concurrent Function Execution Requirements

Your task is to determine how to enforce these synchronization requirements using semaphores. Your solution should not be more restrictive than necessary, and it should ensure that deadlock is not possible.

Using semaphore to solve the problem.

23. Consider the traffic deadlock depicted in the following Figure.



Show that the four necessary conditions for deadlock indeed hold in this example. State a simple rule for avoiding deadlocks in this system.

24. Compare the circular-wait scheme with the deadlock-avoidance schemes (like the banker's algorithm) with respect to the following issues:

- a) Runtime overheads
- b) System throughput

25. Consider a system consisting of m resources of the same type, being shared by n processes. Resources can be requested and released by processes only one at a time. Show that the system is deadlock free if the following two conditions hold:

- a) The maximum need of each process is between 1 and m resources

- b) The sum of all maximum needs is less than $m + n$

26. Consider the following snapshot of a system:

	<u>Allocation</u>	<u>Max</u>
	<i>A B C D</i>	<i>A B C D</i>
P_0	3 0 1 4	5 1 1 7
P_1	2 2 1 0	3 2 1 1
P_2	3 1 2 1	3 3 2 1
P_3	0 5 1 0	4 6 1 2
P_4	4 2 1 2	6 3 2 5

Using the banker's algorithm, determine whether or not each of the following states is unsafe. If the state is safe, illustrate the order in which the threads may complete. Otherwise, illustrate why the state is unsafe.

- a. Available = (0, 3, 0, 1)
b. Available = (1, 0, 0, 2)

27. Consider the following snapshot of a system: Answer the following questions using the banker's algorithm:

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	<i>A B C D</i>	<i>A B C D</i>	<i>A B C D</i>
P_0	2 0 0 1	4 2 1 2	3 3 2 1
P_1	3 1 2 1	5 2 5 2	
P_2	2 1 0 3	2 3 1 6	
P_3	1 3 1 2	1 4 2 4	
P_4	1 4 3 2	3 6 6 5	

- a) Illustrate that the system is in a safe state by demonstrating an order in which the threads may complete.
b) If a request from process P_1 arrives for (1, 1, 0, 0), can the request be granted immediately?
c) If a request from thread P_4 arrives for (0, 0, 2, 0), can the request be granted immediately?

29. 假设一个系统具有如下快照:

	Allocation				Max			
	A	B	C	D	A	B	C	D
T0	1	2	0	2	4	3	1	6
T1	0	1	1	2	2	4	2	4

T2	1	2	4	0	3	6	5	1
T3	1	2	0	1	2	6	2	3
T4	1	0	0	1	3	1	1	2

采用银行家算法，确定如下每个状态是否安全。如果状态是安全的，那么说明进程可以完成的顺序。否则，说明为什么状态是不安全的。

- Available = (2, 2, 2, 3)
- Available = (4, 4, 1, 1)
- Available = (3, 0, 1, 4)
- Available = (1, 5, 2, 2)

30. 假设一个系统具有如下的快照：

	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
T0	3	1	4	1	6	4	7	3	2	2	2	4
T1	2	1	0	2	4	2	3	2				
T2	2	4	1	3	2	5	3	3				
T3	4	1	1	0	6	3	3	2				
T4	2	2	2	1	5	6	7	5				

采用银行家算法，回答下面的问题：

- 通过进程可以完成执行的顺序说明系统处于安全状态。
- 当进程 T4 的请求为 (2, 2, 2, 4) 时，能否立即允许这一请求？
- 当进程 T2 的请求为 (0, 1, 1, 0) 时，能否立即允许这一请求？
- 当进程 T3 的请求为 (2, 2, 1, 2) 时，能否立即允许这一请求？

31. [408] 三个人一起植树，甲挖坑，乙放树苗入坑并填土，丙负责为新种树苗浇水。步骤依次为：挖树坑，放树苗，填土和浇水。现在有铁锹和水桶各一个，铁锹用于挖树坑，填土。水桶用于浇水。当树坑数量小于 3 时，甲才可以挖树坑。设初始坑 = 0，铁锹水桶均可用，定义尽可能少的信号量，用 wait() 和 signal() 操作描述植树过程中三人的同步互斥关系，并说明所用信号量的作用及其初值。

32. [408] 计算机系统进程之间往往需要相互协作以完成一个任务，在某网络系统中缓冲区 B 用于存放一个数据分组，对 B 的操作有 C1、C2 和 C3。C1 将一个数据分组写入 B 中，C2 从 B 中读出一个数据分组，C3 对 B 中的数据分组进行修改。要求 B 为空时才能执行 C1，B 非空时才能执行 C2 和 C3。请回答下列问题。

(1) 假设进程 P1 和 P2 均需执行 C1，实现 C1 的代码是否为临界区？为什么？(2 分)

(2) 假设 B 初始为空，进程 P1 执行 C1 一次，进程 P2 执行 C2 一次。请定义尽可能少的信号量。并用 wait(), signal() 操作描述进程 P1、P2 之间的同步或互斥关系，说明所用信号量的作用及初值。(3 分)

(3) 假设 B 初始不为空，进程 P1 和 P2 各执行 C3 一次，请定义尽可能少的信号量。并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及初值。(3 分)

33. A computer system has 3 types of resources A, B, and C with different numbers of instances. There are 4 running processes P1, P2, P3, P4. The total resources, the resource's Allocation and Max matrices for the four processes are shown as follows:

Process	Allocation	Max
	A B C	A B C
P1	1 3 1	6 5 3
P2	0 2 2	3 5 3
P3	0 2 0	3 5 2
P4	0 1 3	2 4 3
total	6 9 6	

- 1) What are the matrices Need and Available for the system?
- 2) Please check if the system is currently deadlocked. Show your steps clearly.
- 3) At the current state, if P₂ requests additional resources [1, 0, 0], can the request be granted without any possible deadlock? Show your steps clearly.
- 4) At the current state, if P₁ requests additional resources [1, 0, 0], can the request be granted without any possible deadlock? Show your steps clearly.

答案:

n=4,m=3,Allocation and Max are already defined.

Key to Q1: Compute Available and Need

	Need
Process	A B C

	Need
P1	5 2 2
P2	3 3 1
P3	3 3 2
P4	2 3 0
Available	3 3 0

Key to Q2: Safety Alg.

1. Work = Available,
Finish[i] = false, $0 \leq i < n$.
2. Find an i = 4.
3. Work = Work + Allocation, Finish[4] = true

Work: 3 4 3

4. Go to step 2.
5. Find an i = 2.
6. Work = $\langle 3, 6, 5 \rangle$,
Finish[2] = true
7. Go to step 2.
8. Find an i = 3.

Safe sequence: $\langle P_4, P_2, P_3, P_1 \rangle$

Key to Q3 Q4: Resource-Request Alg.

Assume the request is granted, use the Safety Alg. to determine whether the system is in safe state.

34. Consider the following system snapshot using the data structures in the Banker's algorithm, with resources A, B, C, and D, and processes P0 to P4:

	Max				Allocation				Need				Available			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	3	3	3	1	2	1	2								
P1	1	4	1	0	1	1	0	0								
P2	2	1	1	1	0	1	0	1								
P3	5	4	3	3	1	1	2	2								
P4	4	2	6	3	1	2	1	2								
Total Res													2	0	2	0

How many resources of type A, B, C, and D are there?

What are the contents of the Need matrix?

Is the system in a safe state? Why?

If a request from process P2 arrives for additional resources of (0,0,2,0), can the Banker's algorithm grant the request immediately? Why?

4、内存管理（含虚拟内存 / TLB）

1. Which statement about the page size is correct?

- A. A smaller page size leads to less TLB miss.
- B. A smaller page size leads to smaller page table.
- C. A smaller page size leads to less I/O overhead.
- D. A smaller page size leads to smaller internal fragmentation.

2. List the three key pieces of information that are stored in a single page table entry.

Assume pure paging and virtual memory.

A frame number;

Valid-invalid bit;

a bit indicating whether the page is in physical memory or on the disk;

a reference bit;

A modified bit.

3. Consider a demand-paging system with the following time-measured utilizations:

CPU utilization	20%
Paging disk	97.7%
Other I/O devices	5%

Which (if any) of the following will (probably) improve CPU utilization? Explain your answer.

- a. Install a faster CPU.
- b. Install a bigger paging disk.
- c. Increase the degree of multiprogramming.
- d. Decrease the degree of multiprogramming.
- e. Install more main memory.
- f. Install a faster hard disk or multiple controllers with multiple hard disks.
- g. Add prepaging to the page fetch algorithms.
- h. Increase the page size.

4. Assume we have a demand-paged memory. The page table is held in registers. It takes 8 milliseconds to service a page fault if an empty page is available or the replaced page is not modified, and 20 milliseconds if the replaced page is modified. Memory access time is 100 nanoseconds. Assume that the page to be replaced is modified 70 percent of the time. What is the maximum acceptable page-fault rate for an effective access time of no more than 200 nanoseconds?

5. What is the cause of thrashing? How does the system detect thrashing? Once it detects thrashing, what can the system do to eliminate this problem?

a1. (5 pts) List the three key pieces of information that are stored in a single page table entry. Assume pure paging and virtual memory.

a2. Consider the following paging memory system:

There are 4 page table entries (with values of 0xC, 0x2, 0x8, 0x5 for entries 0,1,2,3, respectively). The physical memory is 128 bytes, with frames of 8 bytes each.

(a) (2 pts) How large (the number of bits) is the physical address?

(b) (2 pts) How large is the virtual address?

(c) (2 pts) What is the physical address (in hex) that corresponds to virtual address 0x1D?

(d) (2 pts) What is the physical address (in hex) that corresponds to virtual address 0x03?

a3. Page Replacement

Consider the reference page sequence is 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, and the number of page frame is 4.

(a) How many page faults for FIFO algorithm?

(b) How many page faults for LRU algorithm?

(c) How many page faults for OPT algorithm?

(d) Suppose we decrease page frame to 3, how many page faults for FIFO algorithm?

(e) What phenomenon will occur?

6. In a demand paging system, the page table of a process is shown in Table 1. [10 marks]
(Please write the answers on the answer sheet.)

Table 1. Page table in a demand paging system

Frame number	Valid-Invalid bit
278H	v
106H	v
	i

- The page size is 4KB.
- One memory access time is 100ns.
- One translation look-aside buffer (TLB) access time is 10ns.
- The average page-fault service time is 10^8 ns (including the time to update the TLB and page table).
- The number of page frames for the process is fixed at 2.
- The least recently used (LRU) algorithm and local replacement algorithm are adopted.
- TLB is initially empty.
- TLB is accessed first, and if TLB is not hit, then the page table is accessed (ignore the TLB update time).
- The valid bit is i, indicating that the page is not in memory, resulting in a page fault interrupt. After the page fault interrupt is processed, return to the instruction that generated the page fault interrupt and execute it again.

(1) Please give the physical address for the virtual address 0162H, 1652H and 4687H. If the physical address does not exist, please explain why. [3 marks]

(2) If the virtual address access sequences are 1362H, 2565H, 15A5H, how long does it take to access the above three virtual addresses in sequence? [4 marks]

(3) If the virtual address access sequences are 1362H, 2565H, 15A5H, what is the

physical address of the virtual address 2565H? Please explain why. [3 marks]

(1) [3 marks]

4KB=2¹²B, 页面大小是 12 位

对于十六进制的 3 个逻辑地址, 后 12 位是页内偏移, 前 4 位是页号。(最高位为页号, 低三位为页内偏移)

0162H, 页号=0, 页内偏移 162H, 查询页表, 对应帧号 278H, 所以物理地址 278162H

1652H, 页号=1, 页内偏移 652H, 查询页表, 对应帧号 106H, 所以物理地址 106652H

4687H, 页号=4, 页内偏移 652H, 查询页表, 不存在, 地址越界, 所以物理地址不存在

(2) [4 marks]

1362H, 页号=1, 访问 TLB 10ns, 初始为空, 内存访问页表 100ns, 合成物理地址再访问内存 100ns, 共计 10+100ns+100ns=210ns

2565H: 页号=2, 访问快表 10ns, 落空, 访问页表 100ns 落空, 进行缺页中断处理 10⁸ns, 快表更新, 访问快表 10ns, 合成物理地址后访问主存 100ns, 共计 10ns+100ns+10⁸ns+10ns+100ns=100 000 220ns。

15A5H: 页号=2, 访问快表, 因第一次访问已将该页号放入快表, 因此花费 10ns 便可合成物理地址, 访问主存 100ns, 共计 10ns+100ns=110ns。

(3) [3 marks]

当访问虚拟地址 2565H 时, 产生缺页中断, 进程分配的帧数 2, 已被 278H, 106H 占用。必须从页表中淘汰一个页面, 根据题目的置换算法, 2# 替换 0# (LRU, 1#刚刚被访问) 278565H

7. Consider the reference page sequence is

1, 2, 3, 4, 2, 3, 1, 4, 5, 6, 7, 4, 5, 6, 7,

and the number of page frame is 3.

(a) (4 points) How many page faults for FIFO algorithm? Fill your answer in the blank, and complete the following diagram to show how the page faults are generated.

(b) (4 points) How many page faults for LRU algorithm? Fill your answer in the blank, and complete the following diagram to show how the page faults are generated.

(c) (4 points) How many page faults for OPT algorithm? Fill your answer in the blank, and complete the following diagram to show how the page faults are generated.

(d) (2 points) Which of the three algorithm achieves the worst page fault rate?

(e) (2 points) Use this reference string to describe what is thrashing.

(f) (4 points) With the reference string, how many frames are required at the timing of each reference. Use the working set model (with window size $\Delta = 5$) to provide your reason.

8. What are the benefits and the disadvantages of each of the following?

Consider both the system level and the programmer level.

a. Synchronous and asynchronous communication

- b. Automatic and explicit buffering
- c. Send by copy and send by reference
- d. Fixed-sized and variable-sized messages

9. Given five memory partitions of 100 KB, 500 KB, 200 KB, 300 KB, and 600 KB (in order), how would each of the first-fit, best-fit, and worst-fit algorithms place processes of 212 KB, 417 KB, 112 KB, and 426 KB (in order)? Which algorithm makes the most efficient use of memory?

10. Compare the main memory organization schemes of contiguous-memory allocation, pure segmentation, and pure paging with respect to the following issues:

- a. external fragmentation
- b. internal fragmentation
- c. ability to share code across processes

11. Consider a paging system with the page table stored in memory.

- a. If a memory reference takes 200 nanoseconds, how long does a paged memory reference take?
- b. If we add associative registers, and 75 percent of all page-table references are found in the associative registers, what is the effective memory reference time? (Assume that finding a page-table entry in the associative registers takes zero time, if the entry is there.)

12. Consider the following segment table:

Segment	Base	Length
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

What are the physical addresses for the following logical addresses? a. 0,430

- a. 1,10
- b. 2,500
- c. 3,400
- d. 4,112

13. Consider the hierarchical paging scheme used by the VAX architecture. How many memory operations are performed when an user program executes a memory load operation?

14. Under what circumstances do page faults occur? Describe the actions taken by the operating system when a page fault occurs?

15. A certain computer provides its users with a virtual-memory space of 232 bytes. The computer has 218 bytes of physical memory. The virtual memory is implemented by paging, and the page size is 4096 bytes. A user process generates the virtual address 11123456. Explain how the system establishes the corresponding physical location. Distinguish between software and hardware operations.

16. Consider the page table for a system with 12-bit virtual and physical addresses and 256-byte pages.

Page	Page Frame
0	–
1	2
2	C
3	A
4	–
5	4
6	3
7	–
8	B
9	0

The list of free page frames is D, E, F (that is, D is at the head of the list, E is second, and F is last). A dash for a page frame indicates that the page is not in memory. Convert the following virtual addresses to their equivalent physical addresses in hexadecimal. All numbers are given in hexadecimal.

- a) 9EF
- b) 111
- c) 700
- d) 0FF

17. Consider the following page reference string: 7, 2, 3, 1, 2, 5, 3, 4, 6, 7, 1, 0, 5, 4, 6, 2, 3, 0, 1.

Assuming demand paging with three frames, how many page faults would occur for the following replacement algorithms?

- LRU replacement
- FIFO replacement
- Optimal replacement

• Clock replacement (命中指针不移动)

18. 给定六个内存分区，分别为 100MB、170MB、40MB、205MB、300MB 和 185MB（按顺序），首次适应、最优适应和最差适应算法将如何放置大小为 200MB、15MB、185MB、75MB、175MB 和 80MB（按顺序）的进程？指出哪个请求（如果有）不能得到满足。评论每种算法管理内存的效率。

19. 考虑一个具有 32 位逻辑地址和 8KB 页大小的计算机系统。系统支持高达 1GB 的物理内存。以下各有多少个条目？

- a. 传统的单层页表
- b. 倒置页表

9.25 假设有一个分页系统，其页表在内存中。

- a. 如果内存引用需要 50ns，分页内存的引用需要多长时间？
- b. 如果添加了 TLB，并且所有页表引用的 75%可在 TLB 中发现，那么内存引用的有效时间是多少？（假设所查的页表条目在 TLB 中时，需要 2ns）。

20. 假设某个系统的页表具有 16 位的虚拟和物理地址，每个页面为 4096 字节。当页面被引用时，其引用位会被设置为 1。有一个线程会周期性地将引用位的所有值清零。页帧的破折号表示页面不在内存中。页面置换算法是局部 LRU，所有数值是按十进制提供的。

页面	页帧	引用位
0	9	0
1	—	0
2	10	0
3	15	0
4	6	0
5	13	0
6	8	0
7	12	0
8	7	0
9	—	0

10	5	0
11	4	0
12	1	0
13	0	0
14	–	0
15	2	0

a. 将以下虚拟地址（十六进制）转换为对应的物理地址。你可以提供十六进制或十进制的答案。你还要在页表中为适当的条目设置引用位。

0x621C

0xF0A3

0xBC1A

0x5BAA

0x0BA1

b. 使用上述地址作为指导，提供导致缺页错误的逻辑地址（十六进制）示例。

c. LRU 页面置换算法在解决缺页错误时会从哪一组页面帧中选择？

21. 对以下页面引用串，采用 FIFO、LRU 和最佳（OPT）替换算法：

2, 6, 9, 2, 4, 2, 1, 7, 3, 0, 5, 2, 1, 2, 9, 5, 7, 3, 8, 5
0, 6, 3, 0, 2, 6, 3, 5, 2, 4, 1, 3, 0, 6, 1, 4, 2, 3, 5, 7
3, 1, 4, 2, 5, 4, 1, 3, 5, 2, 0, 1, 1, 0, 2, 3, 4, 5, 0, 1
4, 2, 1, 7, 9, 8, 3, 5, 2, 6, 8, 1, 0, 7, 2, 4, 1, 3, 5, 8
0, 1, 2, 3, 4, 4, 3, 2, 1, 0, 0, 1, 2, 3, 4, 4, 3, 2, 1, 0

假设使用三帧的请求调页，给出每个算法的缺页数量。

22. [408]某进程的虚拟地址空间如图，阴影部分为未占用区域，有 C 程序：

操作系统内核区
用户栈

共享库的存储映射区
动态生成的堆 (运行时由 malloc 创建)
读/写数据段
只读代码段
未使用区

```
char * ptr;
void main() {
    int length;
    ptr=(char*) malloc(100);
    scanf("%s", ptr);
    length = strlen(ptr);
    printf("length=%d\n", length);
    free(ptr) ;
}
```

- (1) 上述程序执行时，PCB 位于哪个区域，执行 scanf () 等待键盘输入时，该进程处于什么状态？
- (2) main() 函数的代码位于哪个区域？其直接调用的哪些函数的功能需要通过执行驱动程序实现？
- (3) 变量 ptr 被分配在哪个区域？若变量 length 没有被分配在寄存器中，则会被分配在哪个区域？ptr 指向的字符串位于哪个区域？

23. [408]某计算机按字节编址，采用页式虚拟存储管理方式，虚拟地址和物理地址的长度均为 32 位，页表项的大小为 4 字节，页大小为 4MB，虚拟地址结构如下：

页号（10 位）	页内偏移量（22 位）
----------	-------------

进程 P 的页表起始虚拟地址为 B8C0 0000H，被装载到从物理地址 6540 0000H 开始的连续主存空间中。请回答下列问题：

- (1) 若 CPU 在执行进程 P 的过程中，访问虚拟地址 1234 5678H 时发生了缺页异常，经过缺页异常处理和 MMU 地址转换后得到的物理地址是 BAB4 5678H。在此

次缺页异常的处理中，需要为新缺页分配页框并更新相应的页表项，则该页表项的虚拟地址和物理地址分别是什么？该页表项中的页框号更新后的值是什么？

(2) 进程 P 的页表所在页的页号是什么？该页对应的页表项的虚拟地址是什么？该页表项中的页框号是多少？

24. Consider a system with 64 MB of physical memory, 32-bit physical addresses, 32-bit virtual addresses, and 4 KB physical page frames.

(a) Using a single-level paging scheme, what is the maximum number of page table entries for a page table in this system?

(b) Using a two-level paging scheme with a 1024-entry outer-page table, how many bits are needed in the logical address to represent the outer page table? How many bits are needed in the logical address in order to represent the inner page table? How many bits are used to represent the offset within a page?

(c) Suppose a TLB is used with the two-level paging scheme described in part (b), and the TLB has a 90% hit rate. If the TLB access time is 10 ns and memory access time is 100 ns, what is the effective memory access time of the system?

答案：

(a) 2^{20}

(b) 10, 10, 12

(c) $EAT = 90\% \times (10 + 100) + 10\% \times (20 + 300) = 99 + 32 = 131$

25. Consider the reference page sequence is 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, and the number of page frame is 3.

(a) How many page faults for FIFO algorithm?

(b) How many page faults for LRU algorithm?

(c) How many page faults for OPT algorithm?

答案：

(a) 9

(b) 10

(c) 7

5、文件系统与外设

1. What are the major activities of an operating system with regard to file management?

2. Some systems provide file sharing by maintaining a single copy of a file; other systems maintain several copies, one for each of the users sharing the file. Discuss the relative merits of each approach.

3. Consider a file currently consisting of 100 blocks. Assume that the file-control block (and the index block, in the case of indexed allocation) is already in memory. Calculate how many disk I/O operations are required for contiguous, linked, and indexed (single-

level) allocation strategies, if, for one block, the following conditions hold. In the contiguous-allocation case, assume that there is no room to grow at the beginning but there is room to grow at the end. Also assume that the block information to be added is stored in memory.

- a) The block is added at the beginning.
- b) The block is added in the middle.
- c) The block is added at the end.
- d) The block is removed from the beginning.
- e) The block is removed from the middle.
- f) The block is removed from the end.

4. Which disk scheduling algorithm could result in starvation?

- A. FCFS
- B. SCAN
- C. SSTF
- D. LOOK

5. Suppose that a disk drive has 5000 cylinders, numbered 0 to 4999. The drive is currently serving a request at cylinder 143, and the previous request was at cylinder 125. The queue of pending requests, in FIFO order, is 86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130.

6. Starting from the current head position, what is the total distance (in cylinders) that the disk arm moves to satisfy all the pending requests, for each of the following disk-scheduling algorithms?

- a. FCFS
- b. SSTF
- c. SCAN
- d. LOOK
- e. C-SCAN

7. Which statement about the file allocation methods is *not* correct?

- A. With linked allocation, there is no external fragmentation.
- B. The indexing structure for indexed allocation wastes disk space.
- C. As a kind of link allocation, FAT allocation can only support sequential-access.
- D. Contiguous allocation is not suitable for files that may dynamically grow in size.

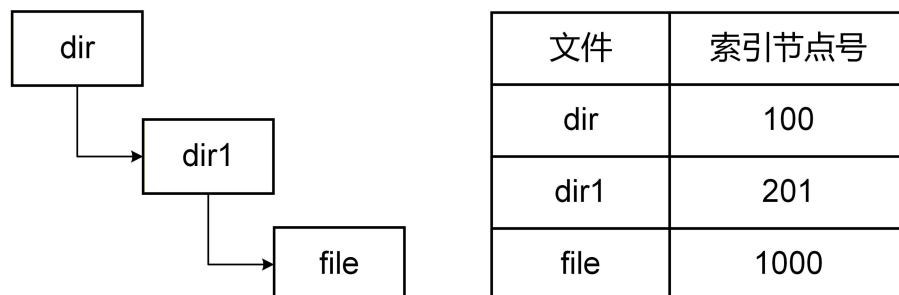
8. Consider a file system where a file can be deleted and its disk space reclaimed while links to that file still exist. What problems may occur if a new file is created in the same storage area or with the same absolute path name? How can these problems be avoided?

9. 考虑一个磁盘的文件系统，其逻辑块和物理块的大小都为 512 字节。假设每个文件的信息已在内存中。针对每种分配策略（连续、链接和索引），回答以下问题：
a. 这个系统的逻辑到物理地址映射是如何实现的？（对于索引分配，假设每个文件总是小于 512 块长。）

b. 如果当前处于逻辑块 10（最后访问的块为块 10）并且需要访问逻辑块 4，必须从磁盘上读取多少物理块？

10. 考虑一个采用 inode 表示文件的文件系统。磁盘块大小为 8KB，磁盘块指针需要 4 字节。这个文件系统具有 12 个直接磁盘块，以及一级、二级和三级间接磁盘块。这个文件系统能存储的文件最大是多少？

11. [408] (本题满分 8 分) 文件系统的目录项包括文件名和索引节点号。磁盘包含索引节点表、位图、目录、文件数据等元数据。若盘块大小为 4 KB，盘块号占 4 B，索引节点表存放了系统的所有文件，从 0 开始编号，存放在盘块号 100 开始连续的 4096 个盘块中。索引节点占用 128 B，包含直接地址项 5 个，一级间接地址项、二级间接地址项、三级间接地址项各 1 个。磁盘位图和索引节点位图分别记录磁盘和索引节点的使用情况，0 表示未使用，1 表示已使用。其中目录结构图与文件的索引节点表如下所示（此处假定图中信息已给出），file 文件占 30 KB。



(1) file 的索引节点所在的盘块号是多少？若 file 的索引节点已经读取到内存，要访问 file 文件中偏移地址 21460 的一个字节数据，则最多需要读多少个盘块？如果文件系统中有足够的磁盘空间，则最多可以存放多少个文件？（3 分）

(2) 如果要删除目录 dir1，则需要对元数据进行哪些操作？（5 分）

14. Suppose that a disk drive has 200 cylinders, numbered 0 to 199. The drive is currently serving a request at cylinder 123, and the previous request was at cylinder 175. The queue of pending requests, in FIFO order, is 86, 147, 180, 28, 95, 151, 12, 77, 30. Starting from the current head position, what is the total distance (in cylinders) that the disk arm moves to satisfy all the pending requests for each of the following disk-scheduling algorithms respectively?

1. FCFS
2. SSTF
3. C-SCAN

4. LOOK